

Implementation Notes SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 Status: Final for Implementation Based On: Folder Structure v1.1, Database Design v1.0, API Design v1.0, AI Pipeline v1.0, Development Roadmap v1.0

1. Purpose

Every prior document specified *what* exists (schemas, endpoints, pipeline stages) and *when* it gets built (Development Roadmap). This document specifies *how the code is actually written* — concrete naming conventions as applied to real files, worked repository/service implementations, and the dependency-injection wiring that connects FastAPI routes to services to repositories to the AI pipeline. It is the reference a developer opens while writing code in Sprint 1, not after Sprint 8.

This is deliberately narrower than a general "Coding Standards" style guide (that document — `docs/standards/Coding_Standards.md` — covers formatting, linting, and PR conventions). This document covers **architecture-in-practice**: SOLID, Repository Pattern, and dependency injection, made concrete rather than left as terminology.

2. Naming Conventions in Practice

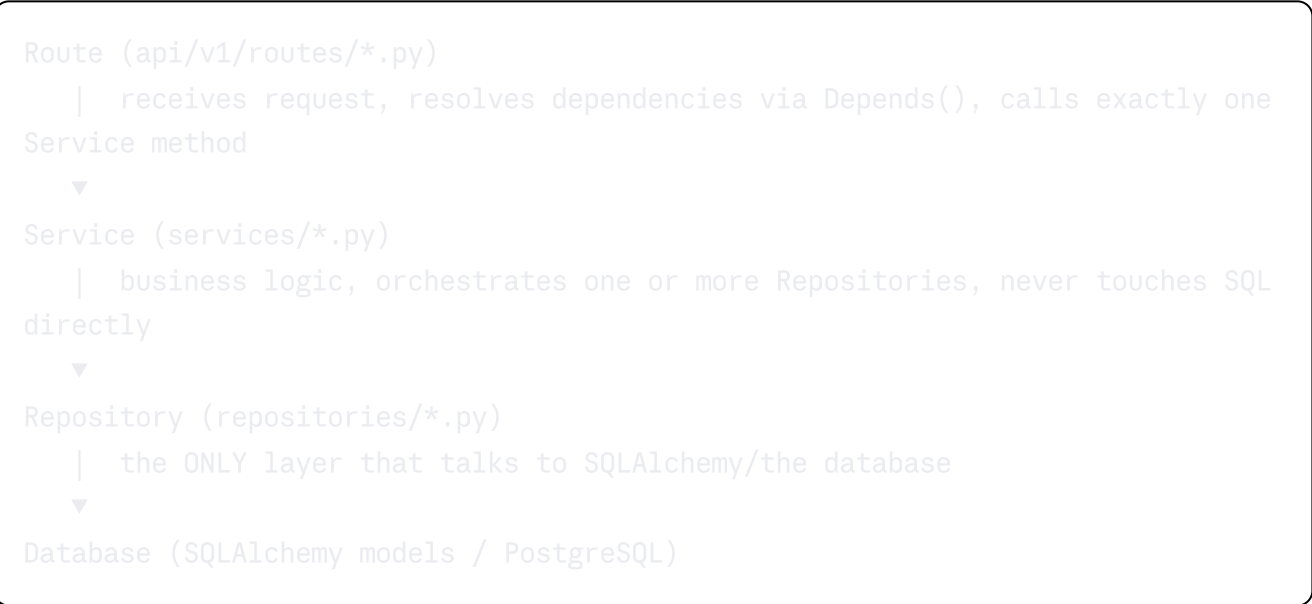
These are not abstract rules — they are the exact conventions used consistently across every code example in this document, so copy-pasting from here into the real codebase requires no translation.

Element	Convention	Example
Python module/file	<code>snake_case.py</code>	<code>detection_repository.py</code>
Python class	<code>PascalCase</code> , suffixed by its architectural role	<code>DetectionRepository</code> , <code>ProfanityDetectionService</code> , <code>FileUploadRequest</code>
Python function/variable	<code>snake_case</code>	<code>get_active_detections</code> , <code>min_confidence</code>
Interface / abstract base class	<code>PascalCase</code> ending in the role name, no <code>I</code> / <code>Abstract</code> prefix (Pythonic, not C#-style)	<code>SpeechRecognitionService</code> (ABC), not <code>ISpeechRecognitionService</code>
Concrete implementation of an interface	<code>PascalCase</code> naming the <i>technology</i> , not a generic suffix like <code>Impl</code>	<code>FasterWhisperEngine</code> implements <code>SpeechRecognitionService</code> — the name tells you <i>which</i> engine at a glance
SQLAlchemy ORM model	Singular noun, matches table name (plural) minus the <code>s</code>	Table <code>files</code> → model class <code>File</code>
Pydantic schema	Suffixed by direction/purpose	<code>FileUploadRequest</code> , <code>FileResponse</code> , <code>DetectionUpdatePatch</code>
FastAPI route function	<code>verb_noun</code> , matches the REST action	<code>create_file</code> , <code>list_files</code> , <code>patch_detection</code>
React component file	<code>PascalCase.jsx</code>	<code>DetectionHighlight.jsx</code>
React hook	<code>camelCase</code> , <code>use</code> prefix	<code>useTimelineEditor.js</code>
Custom exception	<code>PascalCase</code> ending in <code>Error</code>	<code>AudioExtractionError</code> , <code>DuplicatePresetError</code>
Celery task	<code>snake_case</code> , verb-first, matches <code>jobs.job_type</code> value	<code>run_initial_processing</code> , <code>run_regeneration</code>

Element	Convention	Example
Environment variable	UPPER_SNAKE_CASE, module-prefixed where it could collide	ASR_MODEL_SIZE, JWT_ACCESS_TOKEN_EXPIRY_MINUTES

Branch/commit naming (feeds docs/standards/Git_Workflow.md, referenced here since it affects daily practice): feature/<sprint-number>-<short-slug> (e.g., feature/sprint3-timeline-editor-highlights), commit messages in imperative mood (Add DetectionRepository.get_active_detections), referencing the SRS-FR ID where the commit implements a specific requirement (e.g., Implement SRS-FR-TLE-04: manual detection add) — this small habit is what makes the Requirements Traceability Matrix (Document 13 companion) verifiable against real commits rather than reconstructed after the fact.

3. Layering Recap (What Calls What)



The rule that must never be violated: **a Route never imports a Repository directly, and a Service never constructs a database session itself.** Every example below exists to make that rule mechanically easy to follow, not just a convention developers have to remember under deadline pressure.

4. Repository Pattern — Worked Example

4.1 Generic Base Repository

```
python
```

```

# backend/app/repositories/base_repository.py
from typing import TypeVar, Generic, Type
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

ModelType = TypeVar("ModelType")

class BaseRepository(Generic[ModelType]):
    """Generic CRUD so every entity-specific repository isn't reinventing
    get_by_id/list/create/delete. Entity-specific repositories subclass this
    and add only the queries that are actually specific to that entity."""

    def __init__(self, session: AsyncSession, model: Type[ModelType]):
        self.session = session
        self.model = model

    async def get_by_id(self, id: str) -> ModelType | None:
        result = await self.session.execute(
            select(self.model).where(self.model.id == id)
        )
        return result.scalar_one_or_none()

    async def create(self, **kwargs) -> ModelType:
        instance = self.model(**kwargs)
        self.session.add(instance)
        await self.session.flush()
        return instance

    async def delete(self, id: str) -> None:
        instance = await self.get_by_id(id)
        if instance:
            await self.session.delete(instance)

```

4.2 Entity-Specific Repository

python

```

# backend/app/repositories/detection_repository.py
from sqlalchemy import select
from app.models.detection import Detection
from app.repositories.base_repository import BaseRepository

class DetectionRepository(BaseRepository[Detection]):
    """Adds only the queries this app actually needs beyond generic CRUD —
    each one traces to a specific FR from the SRS."""

    def __init__(self, session):
        super().__init__(session, Detection)

    async def get_active_by_file(self, file_id: str) -> list[Detection]:
        # SRS-FR-TLE-01/02: transcript view excludes dismissed detections
        result = await self.session.execute(
            select(Detection)
            .where(Detection.file_id == file_id, Detection.is_dismissed == False)
            .order_by(Detection.start_time)
        )
        return list(result.scalars().all())

    async def bulk_patch(self, updates: list[dict]) -> int:
        # Backs PATCH /files/{id}/detections/batch (API Design §5.4)
        count = 0
        for update in updates:
            detection = await self.get_by_id(update["id"])
            if detection is None:
                continue
            for key, value in update.items():
                if key != "id":
                    setattr(detection, key, value)
            count += 1
        await self.session.flush()
        return count

```

Why this matters beyond style: because `DetectionRepository` is the *only* place `Detection` is queried, a future change (e.g., adding soft-delete semantics, or swapping to a read-replica for list queries) touches one file, not every route handler that happens to need detections — this is the concrete payoff of SRS-NFR-MAINT-01's "modular, layered architecture" requirement, not just a label on an architecture diagram.

5. Service Layer — Worked Example (with Dependency Injection)

python

```

# backend/app/services/timeline_edit_service.py
from app.repositories.detection_repository import DetectionRepository
from app.repositories.file_repository import FileRepository
from app.core.exceptions import NotFoundError, InvalidTimeRangeError

class TimelineEditService:
    """Depends only on repository INTERFACES it needs – never imports
    SQLAlchemy, never imports FastAPI. This is what makes the service
    layer independently unit-testable (Development Roadmap Sprint 8's
    regression pass runs these tests without spinning up a real DB)."""

    def __init__(
        self,
        detection_repo: DetectionRepository,
        file_repo: FileRepository,
    ):
        self.detection_repo = detection_repo
        self.file_repo = file_repo

    async def add_manual_detection(
        self, file_id: str, user_id: str, detected_text: str,
        start_time: float, end_time: float, replacement_mode: str,
    ):
        # Ownership check belongs HERE, not in the route – SRS-NFR-SEC-04 (RBAC)
        file = await self.file_repo.get_by_id(file_id)
        if file is None or file.user_id != user_id:
            raise NotFoundError("File not found")

        # SRS-FR-TLE-04's implicit validation rule (API Design §5.2 error case)
        if end_time <= start_time:
            raise InvalidTimeRangeError("end_time must be after start_time")

        return await self.detection_repo.create(
            file_id=file_id,
            detected_text=detected_text,
            start_time=start_time,
            end_time=end_time,
            replacement_mode=replacement_mode,
            is_user_added=True,
            detection_source="manual",
            confidence_score=1.0, # user-added detections are certain by definition
        )

```

Note on `confidence_score=1.0`: this is a direct, deliberate implementation of the UI/UX Design document's §9.4 rule that manually-added detections render with a distinct solid outline rather than the graded confidence color scale — setting the score to `1.0` here (rather than `null`) keeps the database schema's `NOT NULL` constraint on `confidence_score` (Database Design §4.8) intact without a special-case column, and the frontend distinguishes "manual" from "AI-inferred" via `detection_source`, not via a missing score.

6. Dependency Injection Wiring (FastAPI `Depends`)

This is the piece that actually connects Sections 4–5 to a live HTTP request, without any layer constructing its own dependencies (the concrete meaning of "use dependency injection where appropriate" from the Master Prompt's Coding Standards section).

python

```
# backend/app/dependencies.py
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db_session
from app.repositories.detection_repository import DetectionRepository
from app.repositories.file_repository import FileRepository
from app.services.timeline_edit_service import TimelineEditService

def get_detection_repository(session: AsyncSession = Depends(get_db_session)):
    return DetectionRepository(session)

def get_file_repository(session: AsyncSession = Depends(get_db_session)):
    return FileRepository(session)

def get_timeline_edit_service(
    detection_repo: DetectionRepository = Depends(get_detection_repository),
    file_repo: FileRepository = Depends(get_file_repository),
) -> TimelineEditService:
    return TimelineEditService(detection_repo, file_repo)
```

python

```

# backend/app/api/v1/routes/detection_routes.py
from fastapi import APIRouter, Depends
from app.services.timeline_edit_service import TimelineEditService
from app.dependencies import get_timeline_edit_service
from app.api.deps.auth_deps import get_current_user
from app.schemas.detection_schema import DetectionCreateRequest, DetectionResponse

router = APIRouter()

@router.post("/files/{file_id}/detections", response_model=DetectionResponse, status_code=201)
async def create_detection(
    file_id: str,
    body: DetectionCreateRequest,
    current_user = Depends(get_current_user),  # SRS-NFR-SEC
    service: TimelineEditService = Depends(get_timeline_edit_service),
):
    # The route's ONLY job: unwrap the request, call one service method, return the response
    # No business logic, no DB access, no ownership check lives here — see Section 7.1
    detection = await service.add_manual_detection(
        file_id=file_id,
        user_id=current_user.id,
        detected_text=body.detected_text,
        start_time=body.start_time,
        end_time=body.end_time,
        replacement_mode=body.replacement_mode,
    )
    return DetectionResponse.model_validate(detection)

```

Why this specific wiring, and not a global singleton container: FastAPI's `Depends()` graph is resolved per-request, which means `get_db_session` can yield a fresh `AsyncSession` scoped to exactly one request/transaction — this is what prevents the classic bug of two concurrent requests silently sharing (and corrupting) the same DB session, which matters directly for SRS-NFR-PERF-04's concurrent-job-handling requirement.

7. Wiring the AI Pipeline Into a Celery Task

The AI Pipeline document (Document 9) defined `ProcessingPipeline` as a plain Python class with no FastAPI or Celery awareness — by design, so it stays testable via the CLI proof-of-concept described in Development Roadmap Sprint 1. The Celery task is a thin adapter, not a place to put pipeline logic:

python

```
# backend/app/workers/tasks/processing_task.py
from app.workers.celery_app import celery_app
from sentinelcut_ai.pipeline.processing_pipeline import ProcessingPipeline
from app.repositories.job_repository import JobRepository
from app.db.session import get_sync_session # Celery tasks use sync sessions, not async

@celery_app.task(bind=True, max_retries=3)
def run_initial_processing(self, file_id: str, config: dict):
    session = get_sync_session()
    job_repo = JobRepository(session)
    try:
        pipeline = ProcessingPipeline.build_default() # composes concrete engine
        result = pipeline.run_initial(file_id, config)
        job_repo.mark_completed(file_id, result)
    except Exception as exc:
        # Matches the retry/backoff sequence diagrammed in System Architecture §4
        job_repo.log_error(file_id, stage="unknown", message=str(exc))
        raise self.retry(exc=exc, countdown=2 ** self.request.retries)
```

`ProcessingPipeline.build_default()` is the single place in the codebase where concrete engines are chosen (`FasterWhisperEngine`, the hybrid `ProfanityDetectionService` chain) — exactly one factory function stands between "the interfaces defined in Document 9" and "a real running pipeline," which is what makes the extensibility points in AI Pipeline §17 (swap ASR engine, swap detection model) a one-function change rather than a search-and-replace across the codebase.

8. Custom Exceptions & Error Mapping

```
python

# backend/app/core/exceptions.py
class SentinelCutError(Exception):
    """Base class; never raised directly."""

class NotFoundError(SentinelCutError):
    pass

class InvalidTimeRangeError(SentinelCutError):
    pass

class DuplicateResourceError(SentinelCutError):
    pass
```

python

```
# backend/app/main.py (exception handler registration)
from fastapi import Request
from fastapi.responses import JSONResponse
from app.core.exceptions import NotFoundError, InvalidTimeRangeError, DuplicateRe

@app.exception_handler(NotFoundError)
async def not_found_handler(request: Request, exc: NotFoundError):
    return JSONResponse(status_code=404, content={
        "success": False,
        "error": {"code": "NOT_FOUND", "message": str(exc)}
    })

@app.exception_handler(InvalidTimeRangeError)
async def invalid_range_handler(request: Request, exc: InvalidTimeRangeError):
    return JSONResponse(status_code=400, content={
        "success": False,
        "error": {"code": "VALIDATION_ERROR", "message": str(exc)}
    })
```

This is what makes the API Design document's error-code table (Document 8 §13) actually true in the running system: services raise domain exceptions and never know about HTTP status codes at all; exactly one place (`main.py`'s handler registry) maps a domain exception to the envelope shape and status code the frontend contract promises.

9. Configuration Pattern

python

```
# backend/app/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    redis_url: str
    jwt_secret: str
    jwt_access_token_expiry_minutes: int = 60
    max_upload_size_mb: int = 500
    guest_session_ttl_hours: int = 24

    class Config:
        env_file = ".env"

settings = Settings() # instantiated once, imported everywhere – never re-read p
```

Directly implements SRS-NFR-MAINT-03 ("externalize configuration rather than hardcoding") — every tunable value named in the SRS and AI Pipeline documents (max file size, token expiry, confidence thresholds) has exactly one settings field, never a magic number inlined in a route or service.

10. SOLID Principles, Made Concrete (Not Just Named)

Principle	Where it shows up in the examples above
Single Responsibility	<code>DetectionRepository</code> only queries; <code>TimelineEditService</code> only orchestrates business rules; the route only translates HTTP↔service calls (Section 3)
Open/Closed	<code>BaseRepository</code> is extended (subclassed), never modified, when a new entity needs a repository (Section 4.1→4.2)
Liskov Substitution	Any concrete <code>SpeechRecognitionService</code> (Faster-Whisper today, a cloud API later) must be substitutable behind the same <code>transcribe()</code> signature without callers changing (AI Pipeline §5, §17)
Interface Segregation	<code>SpeechRecognitionService</code> and <code>ProfanityDetectionService</code> are separate interfaces, not one giant <code>AIEngine</code> interface — a service that only needs transcription never depends on detection-scoring methods it doesn't use
Dependency Inversion	<code>TimelineEditService</code> depends on <code>DetectionRepository</code> / <code>FileRepository</code> (concretions injected via <code>Depends()</code>), but nothing in the service layer imports SQLAlchemy directly — the direction of dependency points toward abstractions the service defines its needs against

11. Frontend Equivalent: The `api-client.js` Boundary

The same "one seam, not scattered calls" discipline applies on the frontend:

```
javascript
```

```
// frontend/src/lib/api-client.js
const BASE_URL = process.env.NEXT_PUBLIC_API_URL;

async function request(path, options = {}) {
  const token = getStoredToken(); // from authStore
  const res = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
  });
  const body = await res.json();
  if (!body.success) throw new ApiError(body.error);
  return body.data;
}

export const apiClient = {
  getDetections: (fileId, minConfidence) =>
    request(`/files/${fileId}/detections?min_confidence=${minConfidence ?? 0}`),
  patchDetection: (id, patch) =>
    request(`/detections/${id}`, { method: "PATCH", body: JSON.stringify(patch) })
  // ... one function per endpoint in API Design Document 8, no component calls
};
```

No component under `components/timeline-editor/` ever calls `fetch` directly — every network call goes through `apiClient`, mirroring the backend rule that routes never touch SQL directly. This is what keeps the error-envelope handling (`ApiError`, matching Document 8 §2.3's error shape) in exactly one place instead of duplicated per component.

12. Next Document

Document 13: Testing — Test Plan, Test Cases, and QA Checklist, formalizing the regression pass scheduled in Development Roadmap Sprint 8, and defining exactly how the layering and patterns established in this document are verified (unit tests against services/repositories in isolation per Section 5's note on testability, integration tests against real routes, and end-to-end tests against the full user journeys from the PRD).

End of Implementation Notes